PYTHON FOR DATA PROCESSIN

Importing Libraries in Python

What libraries and modules are, why we import them, every import style, and hands-on examples with NumPy, Pandas, Matplotlib and more

```
import
numpy as np
```

Module Basics • Import Styles • Data-Processing Libraries • Practice

LEARNING OBJECTIVES

By the End of This Topic, You Will Be Able To...

- ✓ Understand what a library is
- ✓ Understand what a module is
- ✓ Import Python libraries in different ways
- ✓ Use built-in and external libraries
- ✓ Understand why libraries are used in data processing
- ✓ Perform basic data processing using Python libraries

LIBRARY BASICS

What Is a Library?

A library is a collection of pre-written code that provides useful, ready-made functions and classes.

Instead of writing everything from scratch, we use libraries to save time and effort.

REAL-LIFE ANALOGY

Think of a library in a college.

Instead of writing your own books, **you borrow books.**

Similarly, instead of writing every function, **you borrow functions from Python libraries.**

```
import math

print(math.sqrt(25))
```

OUTPUT

```
5.0
```

No need to write your own square-root algorithm — math already has it.

LIBRARY BASICS

Beginner

Without a Library vs. With a Library

Suppose you want to calculate a square root. Compare the two approaches:

WITHOUT A LIBRARY

You would have to write the entire algorithm yourself — for example, using Newton's method to approximate a square root:

```
def my_sqrt(n):
    guess = n / 2
    for _ in range(20):
        guess = (guess + n/guess) / 2
    return guess

print(my_sqrt(25))
```

USING PYTHON'S math LIBRARY

```
import math

print(math.sqrt(25))
```

OUTPUT

```
5.0
```

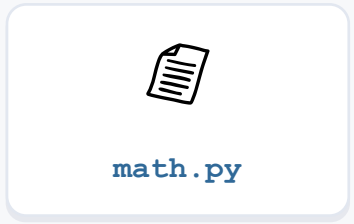
One line. Reliable. Tested.

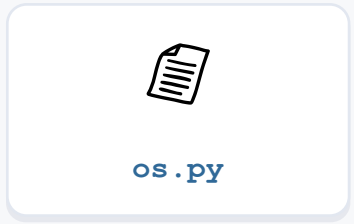
LIBRARY BASICS

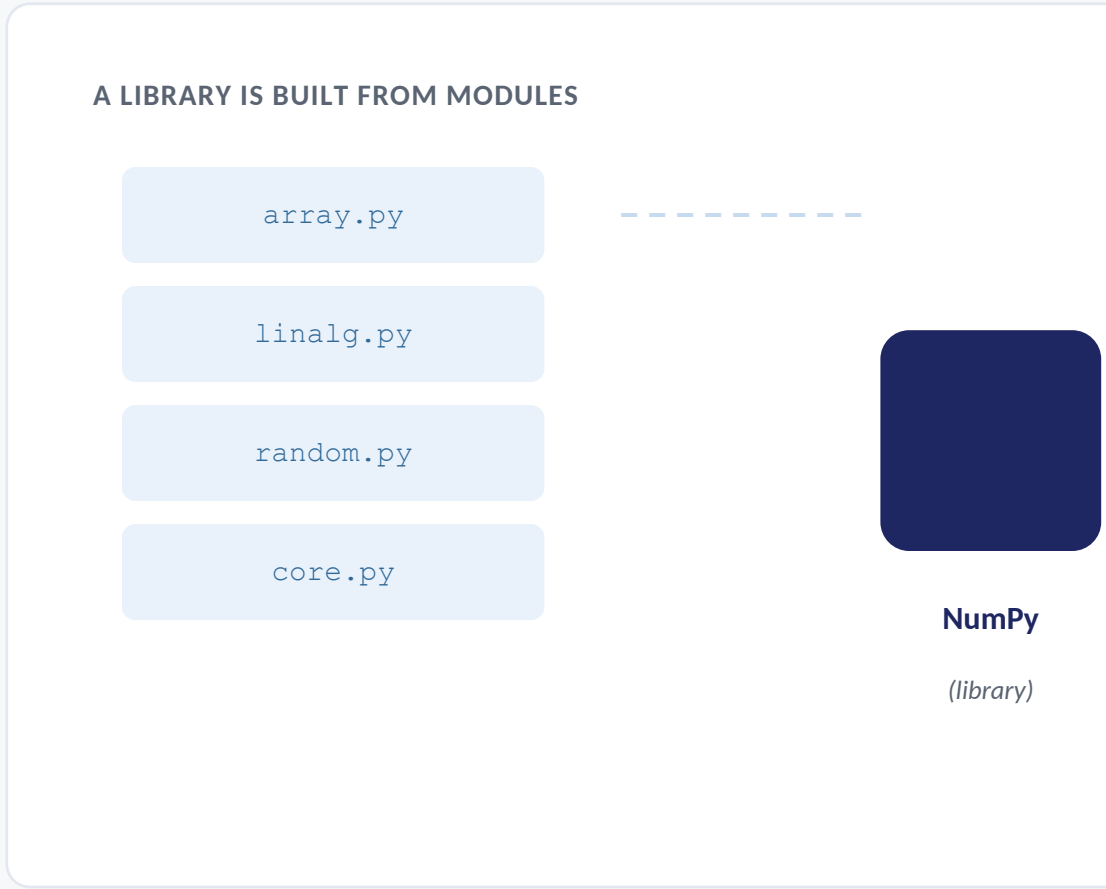
What Is a Module?

A module is a single Python file containing functions and variables.
Many modules together form a library.

Many modules together form a library.







LIBRARY BASICS

Module vs. Library

	Module	Library
Size	Single Python file	Collection of modules
Scope	Small	Large
Example	math	NumPy

A library is essentially a bundle of related modules, packaged together for a bigger purpose (e.g. NumPy bundles many modules for numerical computing).

LIBRARY BASICS

Why Do We Import Libraries?

Libraries provide ready-made functions, giving you these advantages:

1

Saves Time

No need to reinvent common functionality

2

Reduces Effort

Less code for you to write and debug

3

Faster Execution

Libraries are often optimized, sometimes in C

4

Reliable Code

Tested by thousands of developers worldwide

5

Easy to Maintain

Updates and bug fixes come from the library

LIBRARY BASICS

Types of Libraries

BUILT-IN LIBRARIES

Already available in Python — no installation required.

- `math`
- `random`
- `os`
- `datetime`
- `statistics`

EXTERNAL LIBRARIES

Need installation before use.

- `NumPy`
- `Pandas`
- `Matplotlib`
- `Scikit-learn`
- `OpenCV`
- `TensorFlow`

```
pip install pandas
```

IMPORT STYLES

Beginner

Method 1 & 2: Basic Import, Single Function

Method 1 — Import the whole module

```
import math

print(math.sqrt(16))
```

OUTPUT

```
4.0
```

Method 2 — Import one function only

```
from math import sqrt

print(sqrt(36))
```

OUTPUT

```
6.0
```

Beginner

Method 3: Import Multiple Functions

You can import several names from the same module in a single line, separated by commas.

- Only the named functions are brought into your code
- No module prefix is needed — call sqrt() and factorial() directly
- Keeps code short while avoiding a full import *
- Good when you need just a handful of functions from a module

```
from math import sqrt, factorial

print(sqrt(81))
print(factorial(5))
```

OUTPUT

```
9.0
120
```

Method 4: Importing With an Alias

An alias gives a module a shorter nickname with the as keyword — extremely common with data libraries.

- import numpy as np lets you write np instead of numpy
- Standard aliases: numpy → np, pandas → pd, matplotlib.pyplot → plt
- Saves typing across a long script with many calls
- The alias is just a local name — the module itself is unchanged

```
import numpy as np

arr = np.array([1, 2, 3])
print(arr)
```

OUTPUT

```
[1 2 3]
```

Why alias? Writing np.array() everywhere is much easier than numpy.array().

IMPORT STYLES

Moderate

Method 5: Importing Everything (import *)

from module import * brings every public name from a module into your code at once.

- No module prefix needed for any name in that module
- Convenient for very small scripts or quick experiments
- Risky: names from different modules can silently clash
- Not recommended in real projects — prefer explicit imports

```
from math import *

print(sqrt(49))
print(pi)
```

OUTPUT

```
7.0
3.141592653589793
```

Caution: although possible, this is not recommended because many names can conflict.

DATA PROCESSING LIBRARIES

Common Libraries Used in Data Processing

N

NumPy

Numerical computation

P

Pandas

Data analysis

M

Matplotlib

Visualization

S

Statistics

Built-in statistics

R

Random

Random number generation

D

Datetime

Dates & times

O

OS

Operating-system access

NUMPY

Moderate

NumPy: Numerical Computation

NumPy provides fast array operations — the foundation for almost all numerical work in Python.

- import numpy as np is the standard convention
- np.array() converts a Python list into a NumPy array
- Arrays support built-in aggregate methods like .mean()
- NumPy operations run much faster than plain Python loops

```
import numpy as np

numbers = np.array([10, 20, 30, 40])

print(numbers)
print(numbers.mean())
```

OUTPUT

```
[10 20 30 40]
25.0
```

PANDAS

Moderate

Pandas: Data Analysis

Pandas organizes data into DataFrames — labeled tables similar to a spreadsheet.

- import pandas as pd is the standard convention
- A dict of lists becomes columns in a DataFrame
- pd.DataFrame() builds the table from that dict
- Row numbers (0, 1, 2...) are added automatically as the index

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'John'],
    'Marks': [85, 90, 95]
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT

```
    Name  Marks
0  Alice     85
1    Bob     90
2   John     95
```

MATPLOTLIB

Moderate

Matplotlib: Visualization

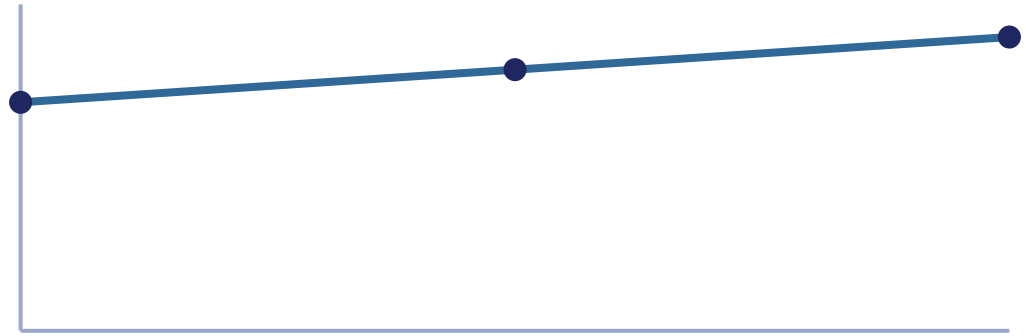
Matplotlib turns numeric data into charts — the standard plotting library for Python.

- import matplotlib.pyplot as plt is the standard convention
- plt.plot() draws a line chart from a list of values
- plt.show() opens a window displaying the chart
- Students will see a line graph rising and falling with the data

```
import matplotlib.pyplot as plt

marks = [70, 80, 90]
plt.plot(marks)
plt.show()
```

RESULTING CHART



STATISTICS & RANDOM

Beginner

statistics and random Modules

statistics — Built-in statistics

```
import statistics

marks = [80, 90, 70, 95]
print(statistics.mean(marks))
```

OUTPUT

```
83.75
```

random — Generate random numbers

```
import random

print(random.randint(1, 100))
```

OUTPUT

```
42

(Output changes
every run.)
```

DATETIME & OS

Beginner

datetime and os Modules

datetime — Dates & times

```
from datetime import datetime

today = datetime.now()
print(today)
```

OUTPUT

```
2026-07-10 10:30:15

(Depends on when
you run it.)
```

os — Operating system access

```
import os

print(os.getcwd())
```

OUTPUT

```
Shows the current
working directory
path.
```

DATA PROCESSING EXAMPLE 1

Moderate

Student Marks with statistics

Combine built-in functions with the statistics module to summarize a data set in three lines.

- max() and min() are built-in Python functions
- statistics.mean() computes the average
- No external installation needed — statistics ships with Python
- This pattern generalizes to any list of numeric data

```
import statistics

marks = [70, 85, 90, 65, 80]

print("Maximum:", max(marks))
print("Minimum:", min(marks))
print("Average:", statistics.mean(marks))
```

OUTPUT

```
Maximum: 90
Minimum: 65
Average: 78.0
```

DATA PROCESSING EXAMPLE 2

Moderate

Array Statistics with NumPy

NumPy arrays come with built-in aggregate methods, making numeric summaries a one-liner each.

- np.mean() computes the average of all elements
- np.sum() adds every element together
- np.max() returns the largest value in the array
- These functions scale efficiently to very large arrays

```
import numpy as np

data = np.array([12, 15, 18, 20, 22])

print("Mean:", np.mean(data))
print("Sum:", np.sum(data))
print("Maximum:", np.max(data))
```

OUTPUT

```
Mean: 17.4
Sum: 87
Maximum: 22
```

DATA PROCESSING EXAMPLE 3

Moderate

Summarizing a DataFrame with Pandas

Once data is in a DataFrame, column-level statistics are a single method call away.

- df['Marks'] selects just the Marks column
- .mean() computes the average of that column
- round(..., 2) keeps the output to two decimal places
- This is the core pattern behind most Pandas analysis

```
import pandas as pd

student = {
    'Name': ['A', 'B', 'C'],
    'Marks': [75, 90, 85]
}

df = pd.DataFrame(student)
print(df)
print(round(df['Marks'].mean(), 2))
```

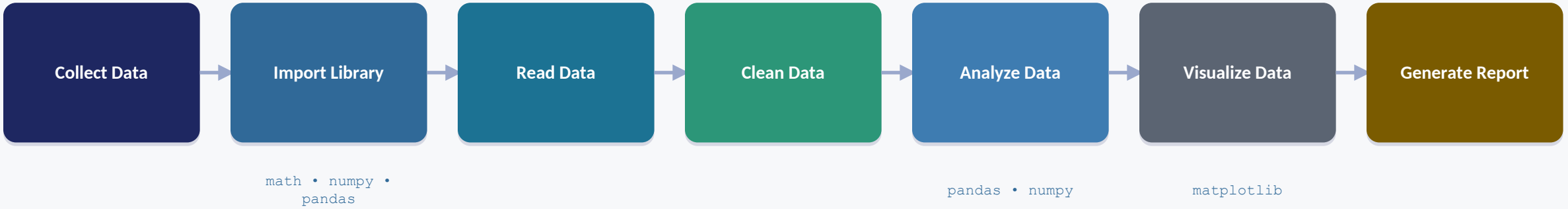
OUTPUT

```
  Name  Marks
0    A     75
1    B     90
2    C     85

83.33
```

WORKFLOW

Data Processing Workflow



Each stage builds on the last: you import the tools you need, then move data through reading, cleaning, analysis, visualization, and finally reporting.

PRACTICE

Activities for Students

Program 1	Import the math library and calculate: square root, power, and factorial.
Program 2	Import the random library. Generate five random numbers.
Program 3	Create a NumPy array and calculate its mean, sum, and maximum.
Program 4	Create a Pandas DataFrame with Name, Age, and Department columns. Display the table.
Program 5	Draw a line chart using Matplotlib.

RECAP

Key Takeaways

1

Libraries & Modules

A library is a collection of modules; a module is a single file of reusable functions and variables.

2

Built-in vs External

Built-in modules (math, random, os) need no install; external libraries (NumPy, Pandas) need pip install.

3

Five Import Styles

import, from-import, multi-import, aliasing (as), and import * — each with its own trade-offs.

4

Data-Processing Libraries

NumPy, Pandas, Matplotlib, statistics, random, datetime and os power the full workflow from data to report.

Thank you!